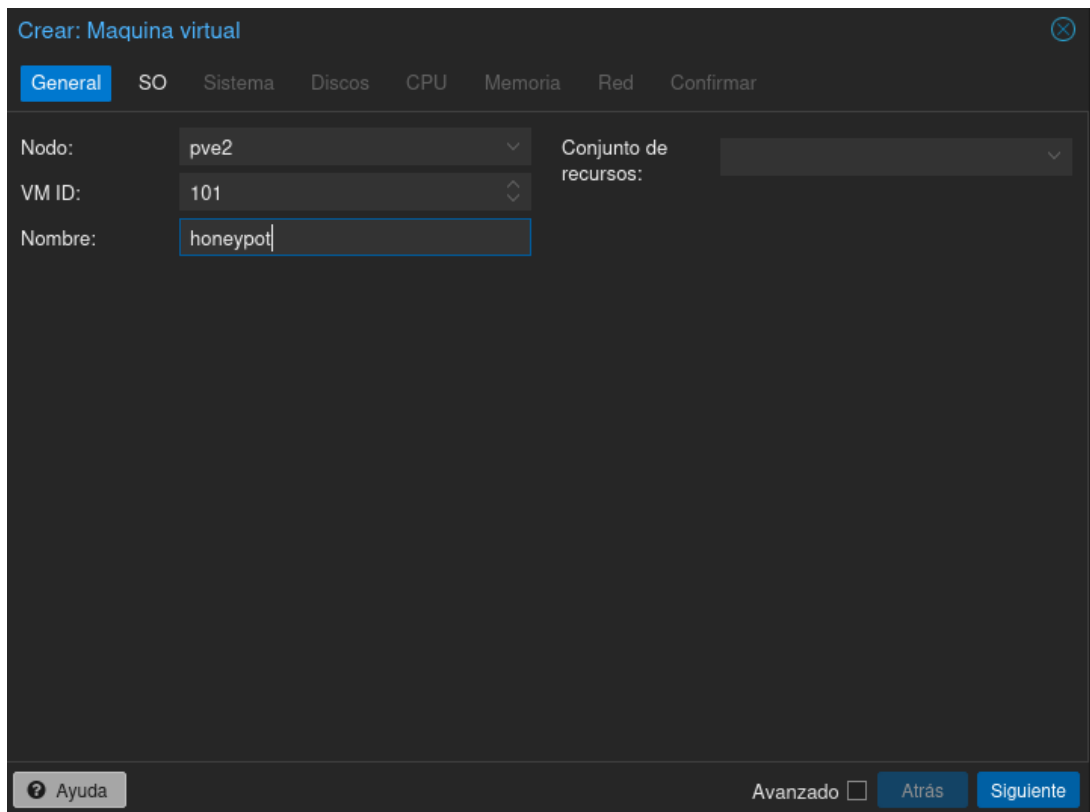


Documentación Técnica - Honeypot Lab

Documentación Técnica — Honeypot Lab

1. Creación de la máquina virtual en Proxmox

1.1 Configuración general



The screenshot shows the 'Crear: Máquina virtual' (Create: Virtual Machine) dialog box in Proxmox VE. The 'General' tab is selected, showing fields for 'Nodo' (Node), 'VM ID', 'Nombre' (Name), and 'Conjunto de recursos' (Resource Set). The 'Nodo' is set to 'pve2', 'VM ID' is '101', and 'Nombre' is 'honeypot'. The 'Conjunto de recursos' is empty. At the bottom, there are buttons for 'Ayuda' (Help), 'Avanzado' (Advanced), 'Atrás' (Back), and 'Siguiete' (Next).

Crear: Máquina virtual

General SO Sistema Discos CPU Memoria Red Confirmar

Nodo: pve2 Conjunto de recursos:

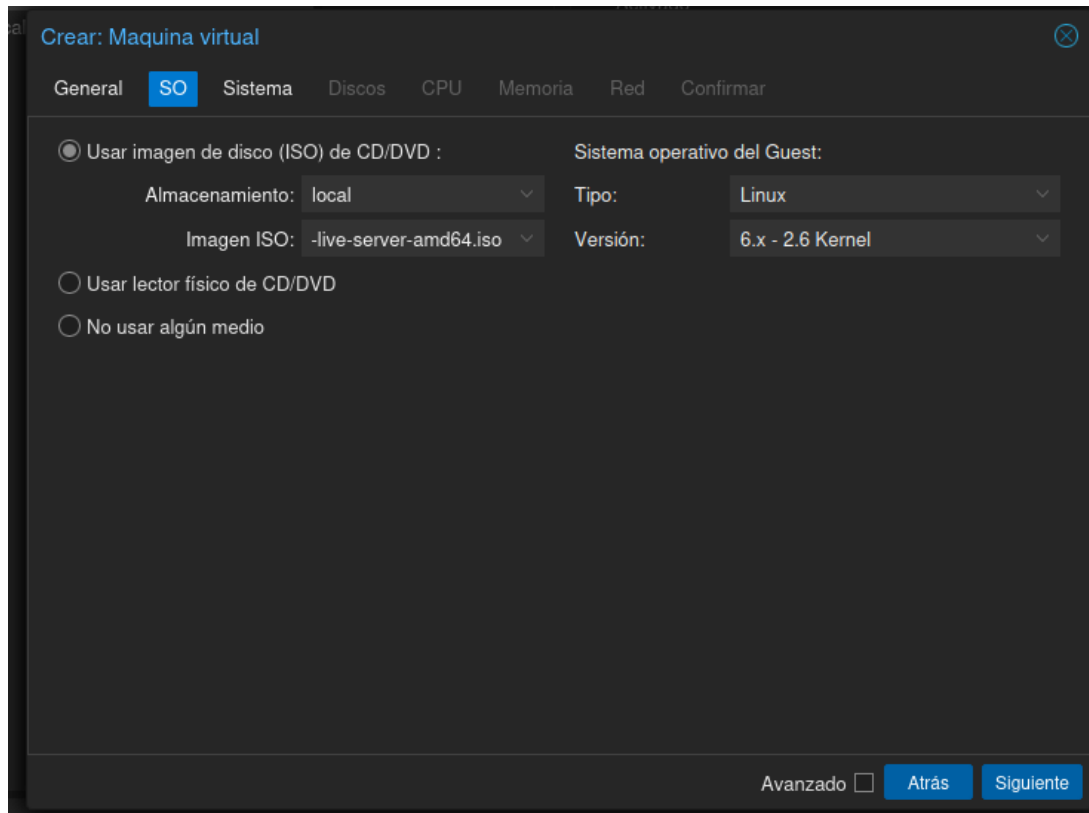
VM ID: 101

Nombre: honeypot

Ayuda Avanzado ☐ Atrás Siguiete

En la interfaz de Proxmox se crea una nueva máquina virtual. Se asigna el nodo **pve2**, el identificador **VM ID 101** y el nombre **honeypot**. Este nombre identifica la máquina dentro del clúster.

1.2 Sistema operativo



Crear: Máquina virtual

General **SO** Sistema Discos CPU Memoria Red Confirmar

☒ Usar imagen de disco (ISO) de CD/DVD :

Almacenamiento: local Tipo: Linux

Imagen ISO: -live-server-amd64.iso Versión: 6.x - 2.6 Kernel

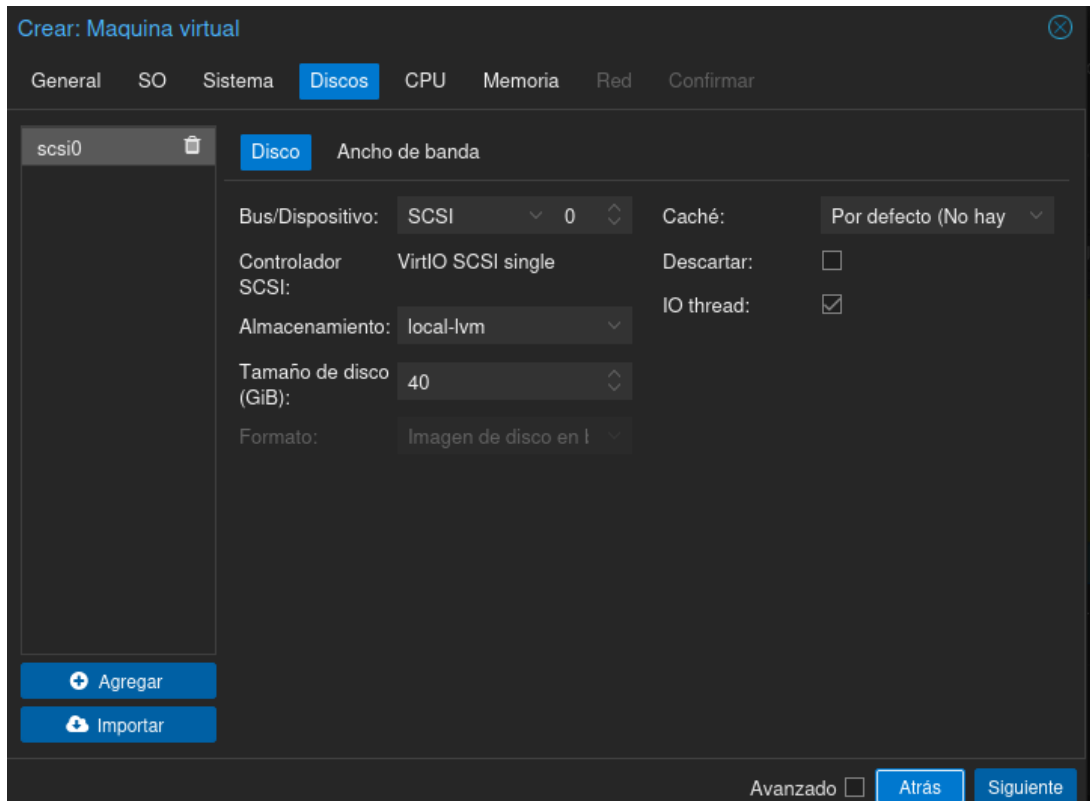
☐ Usar lector físico de CD/DVD

☐ No usar algún medio

Avanzado ☐ Atrás Siguiete

Se selecciona la imagen ISO almacenada en local para instalar el sistema operativo. El tipo de SO configurado es **Linux**, versión de kernel **6.x - 2.6**, usando una imagen live-server para arquitectura amd64.

1.3 Disco duro



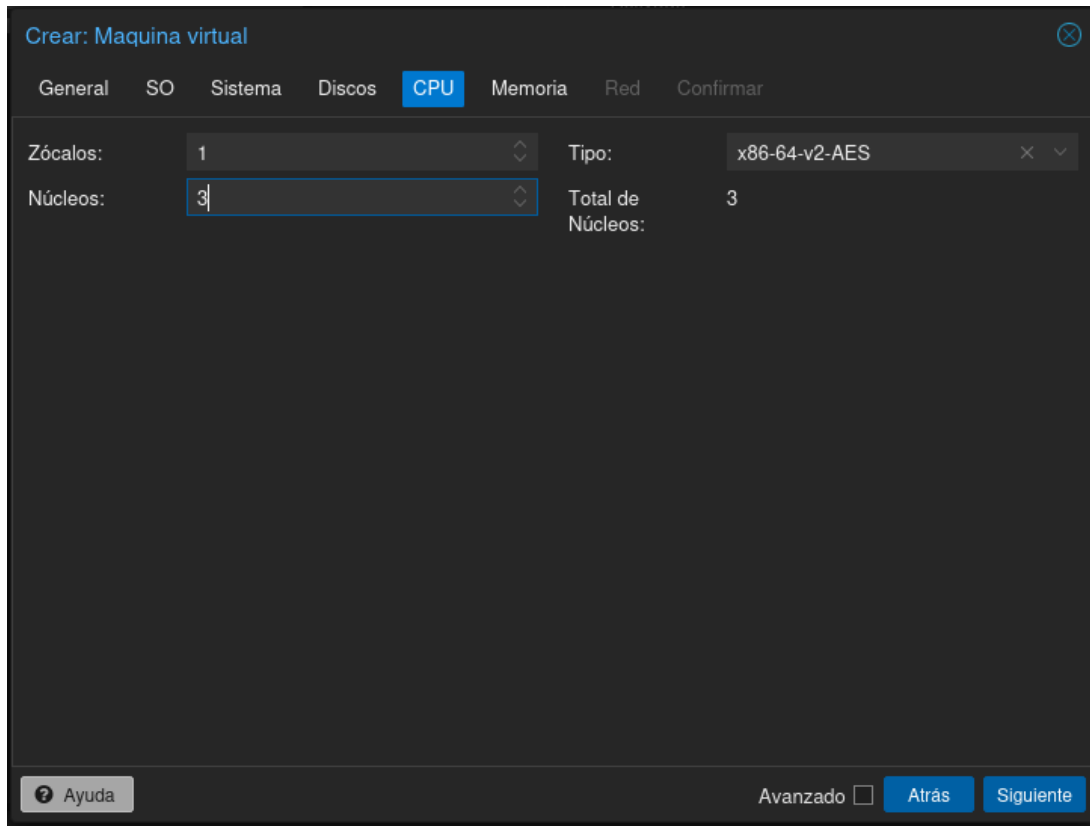
The screenshot shows the 'Crear: Maquina virtual' window with the 'Discos' tab selected. The configuration for the disk is as follows:

Disco	Ancho de banda
scsi0	
Bus/Dispositivo:	SCSI 0
Controlador SCSI:	VirtIO SCSI single
Almacenamiento:	local-lvm
Tamaño de disco (GiB):	40
Formato:	Imagen de disco en l
Caché:	Por defecto (No hay)
Descartar:	☐
IO thread:	☒

Buttons at the bottom: **Agregar**, **Importar**, **Avanzado** (checkbox), **Atrás**, **Siguiente**.

Se configura un disco virtual de **40 GiB** usando el bus SCSI con controlador **VirtIO SCSI single**, almacenado en **local-lvm**. Se activa la opción **IO thread** para mejorar el rendimiento de entrada/salida.

1.4 CPU



The screenshot shows a window titled "Crear: Maquina virtual" with a close button in the top right corner. Below the title bar is a tabbed interface with the following tabs: "General", "SO", "Sistema", "Discos", "CPU" (which is selected and highlighted in blue), "Memoria", "Red", and "Confirmar".

Under the "CPU" tab, there are four fields:

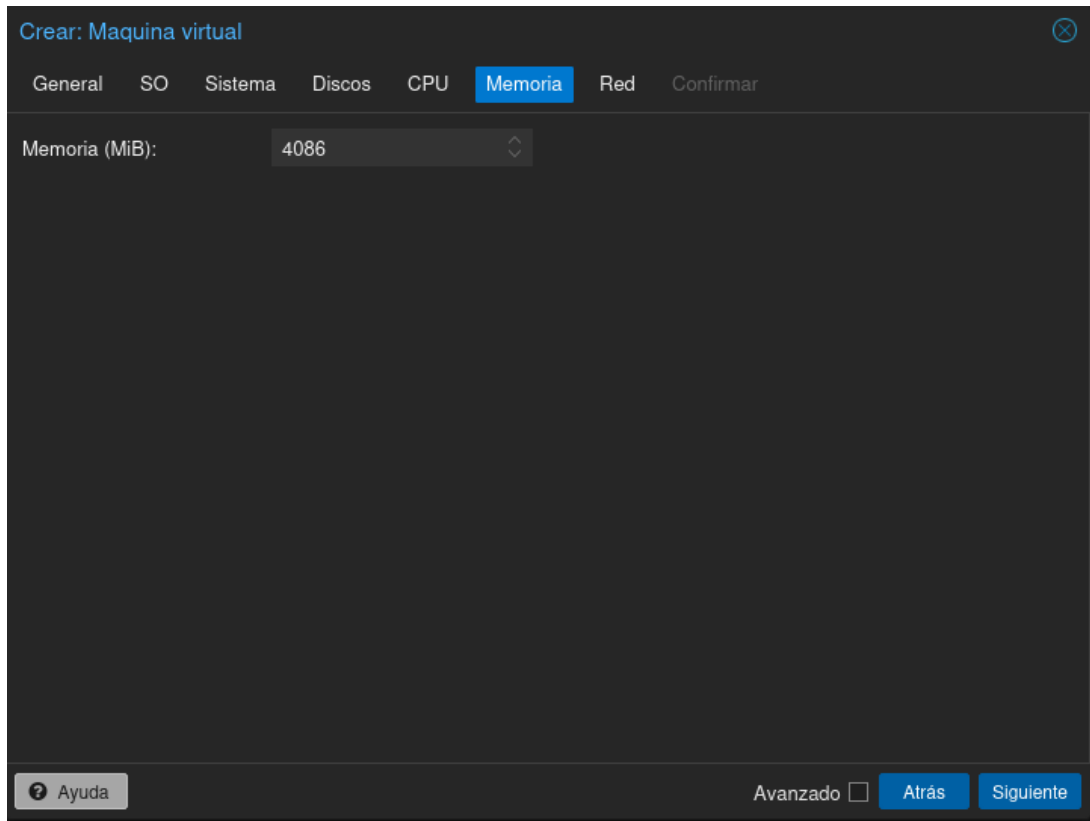
- Zócalos:** A dropdown menu showing the value "1".
- Núcleos:** A dropdown menu showing the value "3".
- Tipo:** A dropdown menu showing the value "x86-64-v2-AES".
- Total de Núcleos:** A text field showing the value "3".

At the bottom of the window, there is a footer area containing:

- An "Ayuda" button with a question mark icon on the left.
- An "Avanzado" checkbox on the right, which is currently unchecked.
- Two blue buttons, "Atrás" and "Siguiente", positioned to the right of the checkbox.

Se asigna **1 zócalo con 3 núcleos** de tipo **x86-64-v2-AES**, resultando en un total de 3 núcleos disponibles para la máquina virtual.

1.5 Memoria RAM



Crear: Máquina virtual

General SO Sistema Discos CPU **Memoria** Red Confirmar

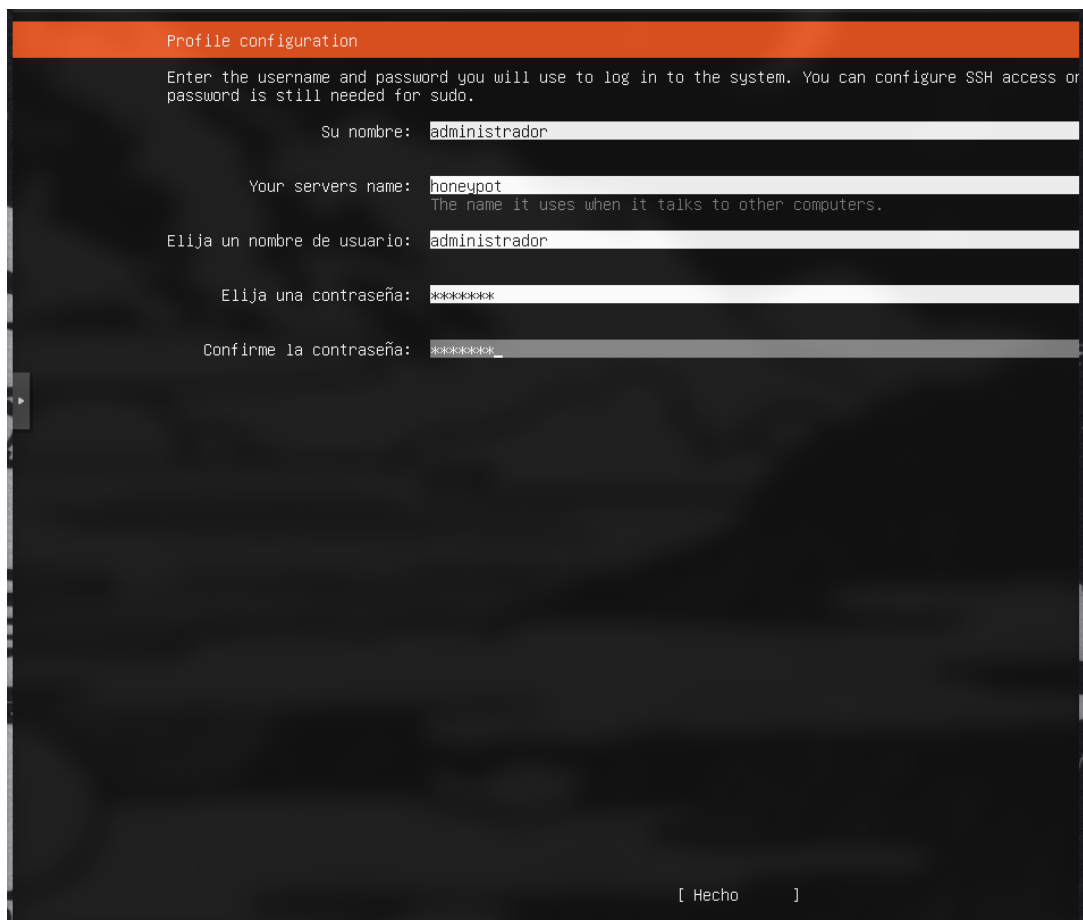
Memoria (MiB): 4086

Ayuda Avanzado ☐ Atrás Siguiente

Se configura **4086 MiB** (aproximadamente 4 GB) de memoria RAM para la máquina virtual.

2. Instalación de Ubuntu Server

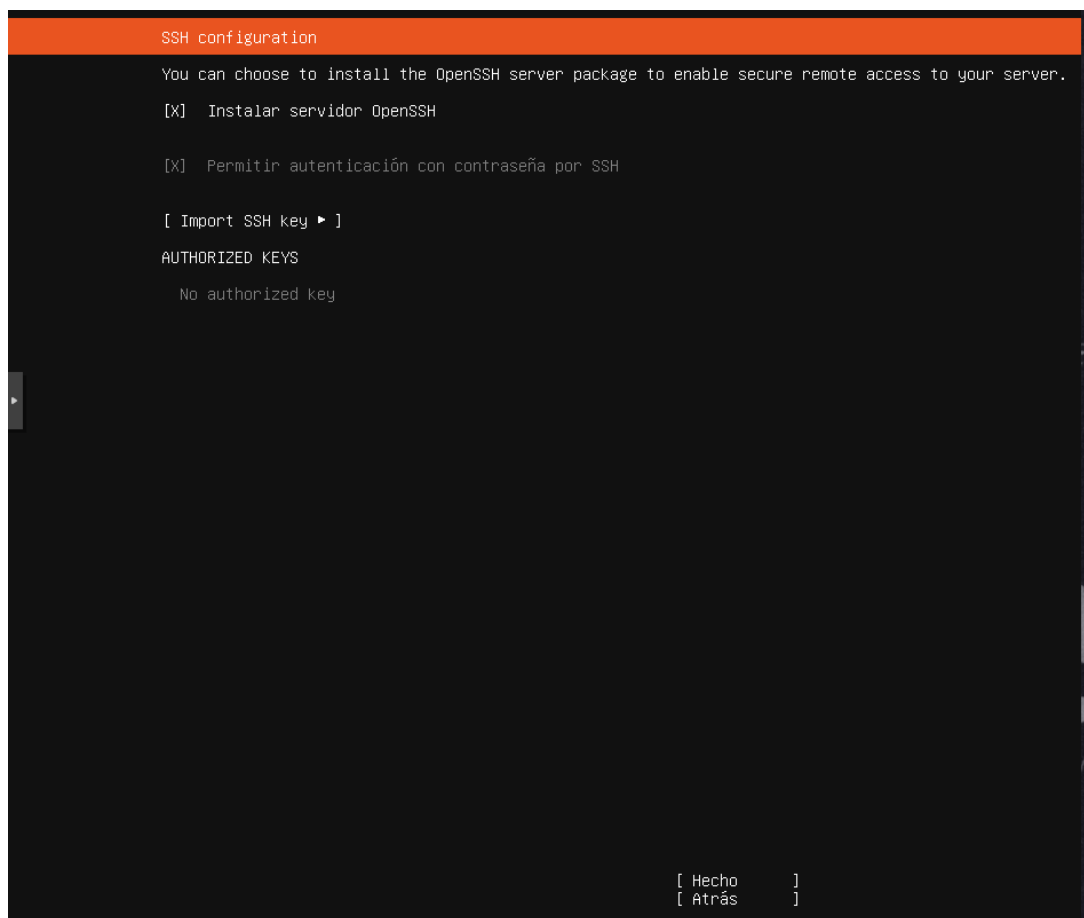
2.1 Configuración del perfil de usuario



The screenshot shows the 'Profile configuration' window during Ubuntu Server installation. It has an orange header bar with the title 'Profile configuration'. Below the header, a message states: 'Enter the username and password you will use to log in to the system. You can configure SSH access or password is still needed for sudo.' The form contains five input fields with labels in Spanish: 'Su nombre:' (Your name) with 'administrador', 'Your servers name:' (The name it uses when it talks to other computers.) with 'honeypot', 'Elija un nombre de usuario:' (Choose a username) with 'administrador', 'Elija una contraseña:' (Choose a password) with '*****', and 'Confirme la contraseña:' (Confirm the password) with '*****'. A right arrow button is on the left side. At the bottom right, there is a button labeled '[Hecho]' (Done).

Durante el proceso de instalación de Ubuntu Server, se configura el perfil del sistema. Se establece el nombre real **administrador**, el nombre del servidor **honeypot**, el nombre de usuario **administrador** y una contraseña. Este usuario tendrá capacidad para ejecutar comandos con sudo.

2.2 Configuración de SSH durante la instalación



En el paso de configuración SSH del instalador se selecciona **instalar el servidor OpenSSH** y se habilita la **autenticación por contraseña**. No se importa ninguna clave SSH adicional. Esto permite acceso remoto a la máquina una vez finalizada la instalación.

3. Configuración del sistema

3.1 Instalación de herramientas del sistema

```
administrador@honeypot:~$ sudo apt install -y \
> git\
> curl\
> wget\
> nano\
> vim\
> htop\
> net-tools\
> build-essential\
> software-properties-common _
```

Desde el terminal del servidor, con el usuario `administrador@honeypot`, se ejecuta `sudo apt install -y` para instalar las siguientes utilidades del sistema: **git**, **curl**, **wget**, **nano**, **vim**, **htop**, **net-tools**, **build-essential** y **software-properties-common**. Estas herramientas son necesarias para gestionar el servidor, compilar dependencias y depurar el entorno.

3.2 Instalación de Python 3.12 dev y venv

```
administrador@honeypot:~$ sudo apt install python3.12-venv python3.12-dev
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  libexpat1-dev libpython3.12-dev python3-pip-whl python3-setuptools-whl zlib1g-dev
Se instalarán los siguientes paquetes NUEVOS:
  libexpat1-dev libpython3.12-dev python3-pip-whl python3-setuptools-whl python3.12-dev python3.12-venv zlib1g-dev
0 actualizados, 7 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 9.643 kB de archivos.
Se utilizarán 35,2 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n]
```

Se instala **python3.12-venv** y **python3.12-dev** mediante apt. El gestor de paquetes resuelve las dependencias necesarias: `libexpat1-dev`, `libpython3.12-dev`, `python3-pip-whl`, `python3-setuptools-whl` y `zlib1g-dev`. En total se descargan 9.6 MB y se utilizan 35.2 MB de espacio en disco.

3.3 Creación del entorno virtual e instalación de dependencias Python

```
administrador@honeypot:~$ python3 -m venv ~/honeypot-env
administrador@honeypot:~$ source ~/honeypot-env/bin/activate
(honeypot-env) administrador@honeypot:~$ pip install --upgrade pip
Requirement already satisfied: pip in ./honeypot-env/lib/python3.12/site-packages (24.0)
Collecting pip
  Downloading pip-26.1.1-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.1.1-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB 12.6 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
Successfully installed pip-26.1.1
(honeypot-env) administrador@honeypot:~$ pip install \
> flask \
> paramiko \
> pycopg2-binary \
> sqlalchemy \
> python-dotenv \
> loguru \
> requests_
```

Se crea el entorno virtual en `~/honeypot-env` con `python3 -m venv` y se activa con `source ~/honeypot-env/bin/activate`. A continuación se actualiza **pip** de la versión 24.0 a la **26.1.1** y se instalan las librerías principales del proyecto: **flask**, **paramiko**, **psycopg2-binary**, **sqlalchemy**, **python-dotenv**, **loguru** y **requests**.

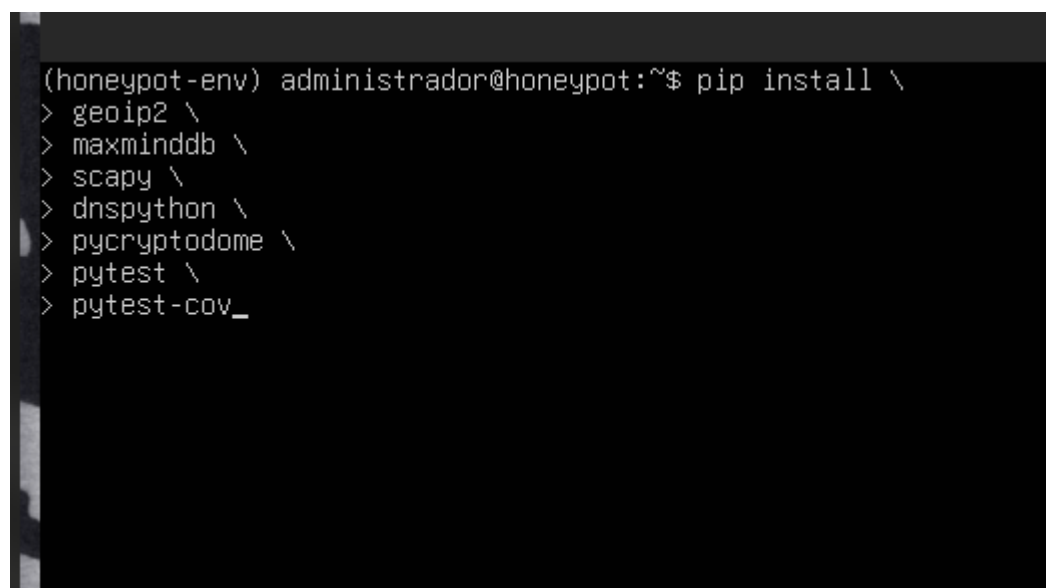
3.4 Verificación de dependencias

```
(honeypot-env) administrador@honeypot:~$ python -c "import flask, paramiko, pycopg2, sqlalchemy, loguru, requests;
todo funciona perfectamente
(honeypot-env) administrador@honeypot:~$ pip list
Package            Version
-----
bcrypt             5.0.0
blinker            1.9.0
certifi            2026.5.20
cffi               2.0.0
charset-normalizer 3.4.7
click              8.4.1
cryptography       48.0.0
flask              3.1.3
greenlet           3.5.1
idna               3.16
invoke             3.0.3
itsdangerous       2.2.0
jinja2             3.1.6
loguru             0.7.3
MarkupSafe         3.0.3
paramiko           5.0.0
pip               26.1.1
psycopg2-binary    2.9.12
pycparser          3.0
PyNaCl            1.6.2
python-dotenv      1.2.2
requests           2.34.2
SQLAlchemy         2.0.49
typing_extensions  4.15.0
urllib3            2.7.0
werkzeug           3.1.8
(honeypot-env) administrador@honeypot:~$ deactivate
administrador@honeypot:~$ ^C
administrador@honeypot:~$
```

Se ejecuta

`python -c "import flask, paramiko, psycpg2, sqlalchemy, loguru, requests"` y el resultado es **“todo funciona perfectamente”**. A continuación se lista con `pip list` todos los paquetes instalados en el entorno: flask 3.1.3, paramiko 5.0.0, psycpg2-binary 2.9.12, SQLAlchemy 2.0.49, loguru 0.7.3, requests 2.34.2, entre otros.

3.5 Instalación de dependencias adicionales

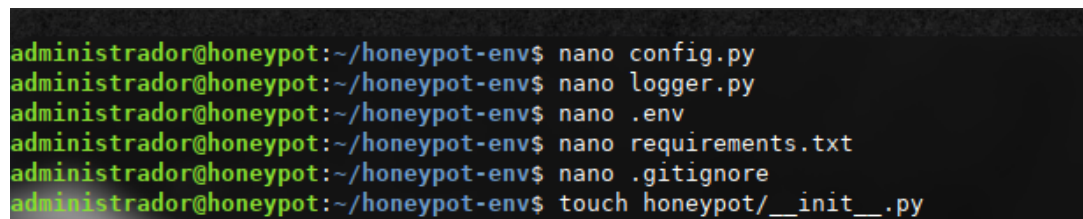


```
(honeypot-env) administrador@honeypot:~$ pip install \  
> geoip2 \  
> maxminddb \  
> scapy \  
> dnspython \  
> pycryptodome \  
> pytest \  
> pytest-cov_
```

Se instalan paquetes adicionales orientados a análisis de red y testing: **geoip2**, **maxminddb**, **scapy**, **dnspython**, **pycryptodome**, **pytest** y **pytest-cov**. Estas librerías amplían las capacidades del honeypot para análisis de tráfico y ejecución de tests automatizados.

4. Estructura del proyecto

4.1 Creación de ficheros de configuración



```
administrador@honeypot:~/honeypot-env$ nano config.py  
administrador@honeypot:~/honeypot-env$ nano logger.py  
administrador@honeypot:~/honeypot-env$ nano .env  
administrador@honeypot:~/honeypot-env$ nano requirements.txt  
administrador@honeypot:~/honeypot-env$ nano .gitignore  
administrador@honeypot:~/honeypot-env$ touch honeypot/__init__.py
```

Con el editor nano se crean los ficheros principales del proyecto dentro del directorio ~/honeypot-env: **config.py**, **logger.py**, **.env**, **requirements.txt** y **.gitignore**. Además se crea el fichero honeypot/ __init__.py para convertir el directorio en un módulo Python.

4.2 Organización de módulos y traslado de ficheros

```
administrador@honeypot: ~/honeypot-lab$ touch honeypot/__init__.py
touch dashboard/__init__.py
touch tests/__init__.py
administrador@honeypot:~/honeypot-lab$ cd
administrador@honeypot:~$ mv honeypot-env/config.py honeypot-lab/
administrador@honeypot:~$ mv honeypot-env/logger.py honeypot-lab/
administrador@honeypot:~$ mv honeypot-env/.env honeypot-lab/
administrador@honeypot:~$ mv honeypot-env/requirements.txt honeypot-lab/
administrador@honeypot:~$ mv honeypot-env/.gitignore honeypot-lab/
administrador@honeypot:~$ _
```

Se crean los ficheros __init__.py en los directorios **honeypot/**, **dashboard/** y **tests/**, estableciendo la estructura de módulos del proyecto. Posteriormente se mueven todos los ficheros de configuración desde ~/honeypot-env/ al directorio definitivo ~/honeypot-lab/ usando el comando mv.

5. Base de datos PostgreSQL

5.1 Instalación de PostgreSQL

```
administrador@honeypot:~/honeypot-lab$ sudo apt install postgresql postgresql-contrib
[sudo] password for administrador:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  libcommon-sense-perl libjson-perl libjson-xs-perl libllvm17t64 libpq5 libtypes-serialiser-perl postgresql-16 postgresql-common ssl-cert
Paquetes sugeridos:
  postgresql-doc postgresql-doc-16
Se instalarán los siguientes paquetes NUEVOS:
  libcommon-sense-perl libjson-perl libjson-xs-perl libllvm17t64 libpq5 libtypes-serialiser-perl postgresql postgresql-client-common postgresql-common postgresql-contrib ssl-cert
0 actualizados, 13 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 43,6 MB de archivos.
Se utilizarán 175 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n]
```

Se instala **postgresql** y **postgresql-contrib** con `sudo apt install`. El gestor de paquetes descarga 43.6 MB e instala 13 paquetes nuevos incluyendo las librerías cliente, el servidor principal y las extensiones adicionales.

5.2 Arranque y habilitación del servicio

```
administrador@honeypot:~/honeypot-lab$ sudo systemctl start postgresql
administrador@honeypot:~/honeypot-lab$ sudo systemctl enable postgresql
Synchronizing state of postgresql.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable postgresql
administrador@honeypot:~/honeypot-lab$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Sun 2026-05-24 20:18:26 UTC; 48s ago
     Main PID: 21536 (code=exited, status=0/SUCCESS)
        CPU: 1ms

May 24 20:18:26 honeypot systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
May 24 20:18:26 honeypot systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
administrador@honeypot:~/honeypot-lab$
```

Se inicia el servicio con `sudo systemctl start postgresql` y se habilita para que arranque automáticamente con `sudo systemctl enable postgresql`. Al consultar el estado con `systemctl status postgresql`, se confirma que el servicio está en estado **active (exited)** con preset habilitado, arrancado el 2026-05-24 a las 20:18:26 UTC.

5.3 Creación de la base de datos

```
administrador@honeypot:~/honeypot-lab$ sudo -u postgres psql
psql (16.14 (Ubuntu 16.14-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# CREATE DATABASE honeypot_db;
CREATE DATABASE
```

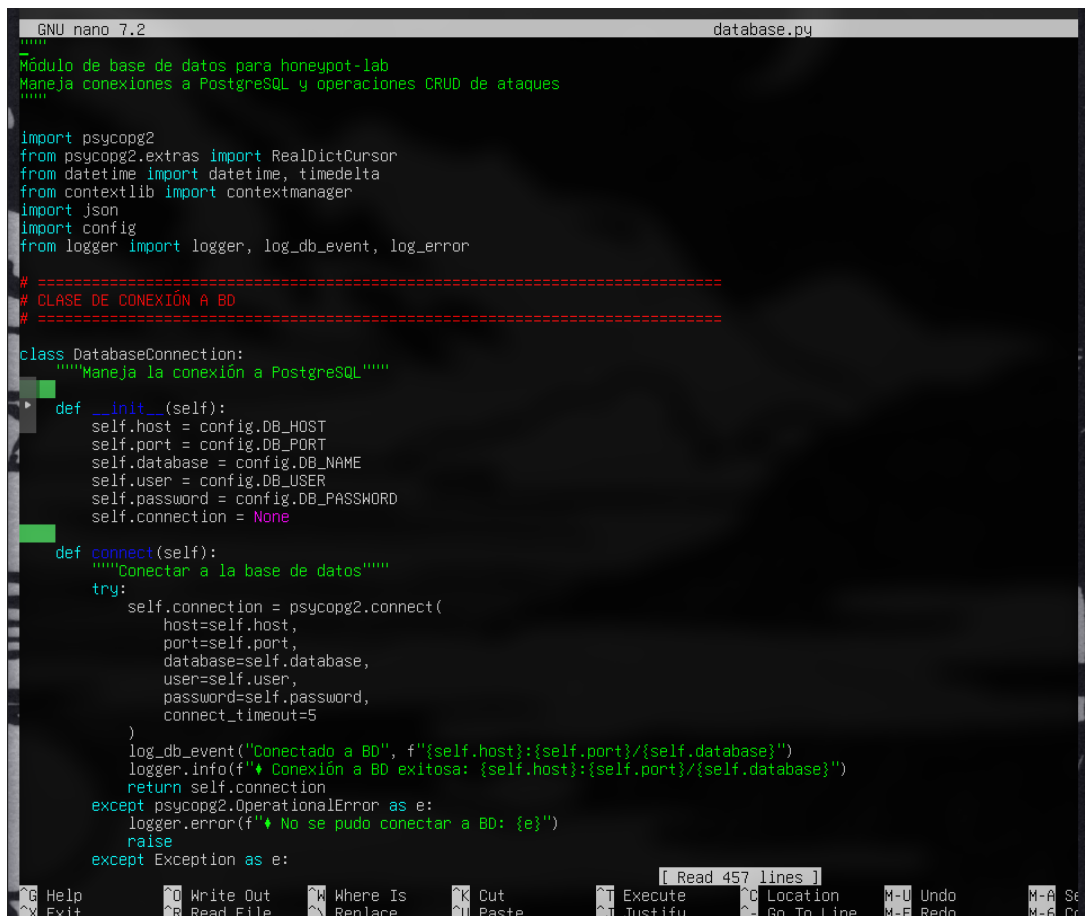
Accediendo al cliente `psql` con `sudo -u postgres psql`, se ejecuta el comando `CREATE DATABASE honeypot_db;`. PostgreSQL confirma la operación con el mensaje **CREATE DATABASE**. La versión instalada es PostgreSQL 16.14 (Ubuntu 16.14-0ubuntu0.24.04.1).

5.4 Creación del usuario y asignación de privilegios

```
postgres=# CREATE USER honeypot_user WITH PASSWORD 'ejemplo';
CREATE ROLE
postgres=# ALTER ROLE honeypot_user SET client_encoding TO 'utf8';
ALTER ROLE
postgres=# GRANT ALL PRIVILEGIES ON DATABASE honeypot_db TO honeypot_user;
ERROR:  syntax error at or near "PRIVILEGIES"
LINE 1: GRANT ALL PRIVILEGIES ON DATABASE honeypot_db TO honeypot_us...
                        ^
postgres=# GRANT ALL PRIVILEGES ON DATABASE honeypot_db TO honeypot_user;
GRANT
postgres=# _
```

Se crea el usuario de base de datos con `CREATE USER honeypot_user WITH PASSWORD 'ejemplo';` — PostgreSQL responde **CREATE ROLE**. Se ajusta la codificación del cliente con `ALTER ROLE honeypot_user SET client_encoding TO 'utf8'`. Se produce un error tipográfico al escribir `PRIVILEGIES` en lugar de `PRIVILEGES`, que se corrige en la siguiente sentencia: `GRANT ALL PRIVILEGES ON DATABASE honeypot_db TO honeypot_user;` — el servidor confirma con **GRANT**.

5.5 Módulo de conexión a base de datos — database.py



```
GNU nano 7.2 database.py
#####
Módulo de base de datos para honeypot-lab
Maneja conexiones a PostgreSQL y operaciones CRUD de ataques
#####

import psycopg2
from psycopg2.extras import RealDictCursor
from datetime import datetime, timedelta
from contextlib import contextmanager
import json
import config
from logger import logger, log_db_event, log_error

# =====
# CLASE DE CONEXIÓN A BD
# =====

class DatabaseConnection:
    """Maneja la conexión a PostgreSQL"""
    def __init__(self):
        self.host = config.DB_HOST
        self.port = config.DB_PORT
        self.database = config.DB_NAME
        self.user = config.DB_USER
        self.password = config.DB_PASSWORD
        self.connection = None

    def connect(self):
        """Conectar a la base de datos"""
        try:
            self.connection = psycopg2.connect(
                host=self.host,
                port=self.port,
                database=self.database,
                user=self.user,
                password=self.password,
                connect_timeout=5
            )
            log_db_event("Conectado a BD", f"{self.host}:{self.port}/{self.database}")
            logger.info(f"♦ Conexión a BD exitosa: {self.host}:{self.port}/{self.database}")
            return self.connection
        except psycopg2.OperationalError as e:
            logger.error(f"♦ No se pudo conectar a BD: {e}")
            raise
        except Exception as e:
            raise
```

Se muestra en el editor nano el fichero **database.py**, que contiene la clase **DatabaseConnection** responsable de gestionar la conexión a PostgreSQL mediante **psycopg2**. La clase lee la configuración del host, puerto, nombre de BD, usuario y contraseña desde el módulo **config**. El método **connect()** establece la conexión con un timeout de 5 segundos, registra el evento con **log_db_event** y lanza una excepción si la conexión falla.

6. SSH Honeypot

6.1 Código del módulo `ssh_trap.py`

```
GNU nano 7.2 database.py
#####
SSH Honeypot - Captura y analiza intentos de login SSH
Simula un servidor SSH falso para detectar ataques de fuerza bruta
Guarda todos los ataques en PostgreSQL
#####

import socket
import threading
import time
import paramiko
import os
import sys
from datetime import datetime

# Agregar parent directory al path para importar módulos
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from logger import logger, log_attack, log_connection, log_error
import config
import database

# CONFIGURACIÓN SSH
# =====

# Generar key RSA para el servidor (simulado)
RSA_KEY = paramiko.RSAKey.generate(config.SSH_KEY_SIZE)

class SSHHoneypot(paramiko.ServerInterface):
    """
    Interfaz de servidor SSH que finge ser un servidor real
    pero captura todos los intentos de login
    """

    def __init__(self, client_address):
        self.client_ip = client_address[0]
        self.client_port = client_address[1]
        self.auth_attempts = 0
        self.username = None
        self.password = None

        # Loguear conexión
        log_connection(self.client_ip, "SSH", config.HONEYPOT_SSH_PORT)
        logger.info(f"↗ Conexión SSH recibida desde {self.client_ip}:{self.client_port}")

[ Read 369 lines ]
G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo     M-A Se
X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo     M-B Co
```



```
GNU nano 7.2 ssh_trap.py
#####
SSH Honeypot - Captura y analiza intentos de login SSH
Emula un servidor SSH falso para detectar ataques de fuerza bruta
Guarda todos los ataques en PostgreSQL
#####

import socket
import threading
import time
import paramiko
import os
import sys
from datetime import datetime

# Agregar parent directory al path para importar módulos
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from logger import logger, log_attack, log_connection, log_error
import config
import database

# =====
# CONFIGURACIÓN SSH
# =====

# Generar key RSA para el servidor (simulado)
RSA_KEY = paramiko.RSAKey.generate(config.SSH_KEY_SIZE)

class SSHHoneypot(paramiko.ServerInterface):
    #####
    Interfaz de servidor SSH que finge ser un servidor real
    pero captura todos los intentos de login
    #####

    def __init__(self, client_address):
        self.client_ip = client_address[0]
        self.client_port = client_address[1]
        self.auth_attempts = 0
        self.username = None
        self.password = None

        # Loguear conexión
        log_connection(self.client_ip, "SSH", config.HONEYPOT_SSH_PORT)
        logger.info(f"🔥 Conexión SSH recibida desde {self.client_ip}:{self.client_port}")

[ Read 346 lines ]
Help Write Out Where Is Cut Execute Location M-U Undo M-A Se
Exit Read File Replace U Paste Justify Go To Line M-E Redo M-B Co
```

Se muestra el código del fichero **ssh_trap.py** abierto en nano. El módulo implementa un servidor SSH falso usando la librería **paramiko**. Al inicio genera una clave RSA para el servidor simulado. La clase SSHHoneypot hereda de paramiko.ServerInterface y actúa como si fuera un servidor SSH real, pero registra cada conexión entrante: almacena la IP del cliente, el puerto, el número de intentos de autenticación y las credenciales probadas. Cada conexión se registra con log_connection al instanciarse.

6.2 Arranque del SSH Honeypot

```
(honeypot-env) administrador@honeypot:~/honeypot-lab$ source ~/honeypot-env/bin/activate
(honeypot-env) administrador@honeypot:~/honeypot-lab$ python3 ssh_trap.py
INFO | __main__:main:295 - =====
INFO | __main__:main:296 - HONEYPOT-LAB - SSH HONEYPOT v0.1 (CON BD)
INFO | __main__:main:297 - =====
INFO | __main__:main:298 - Inicializando SSH Honeypot...
INFO | __main__:main:299 - Host: 0.0.0.0
INFO | __main__:main:300 - Puerto: 2222
INFO | __main__:main:301 - Banner: SSH-2.0-OpenSSH_7.4
INFO | __main__:main:302 - Max intentos auth: 3
INFO | __main__:main:303 - =====
INFO | __main__:main:306 -
♦ Inicializando base de datos...
INFO | database:connect:39 - ♦ Conexión a BD exitosa: localhost:5432/honeypot_db
INFO | database:init_database:408 - ♦ Tabla 'attacks' inicializada correctamente
INFO | logger:log_db_event:50 - Tabla inicializada | attacks
INFO | __main__:main:309 - ♦ Base de datos lista

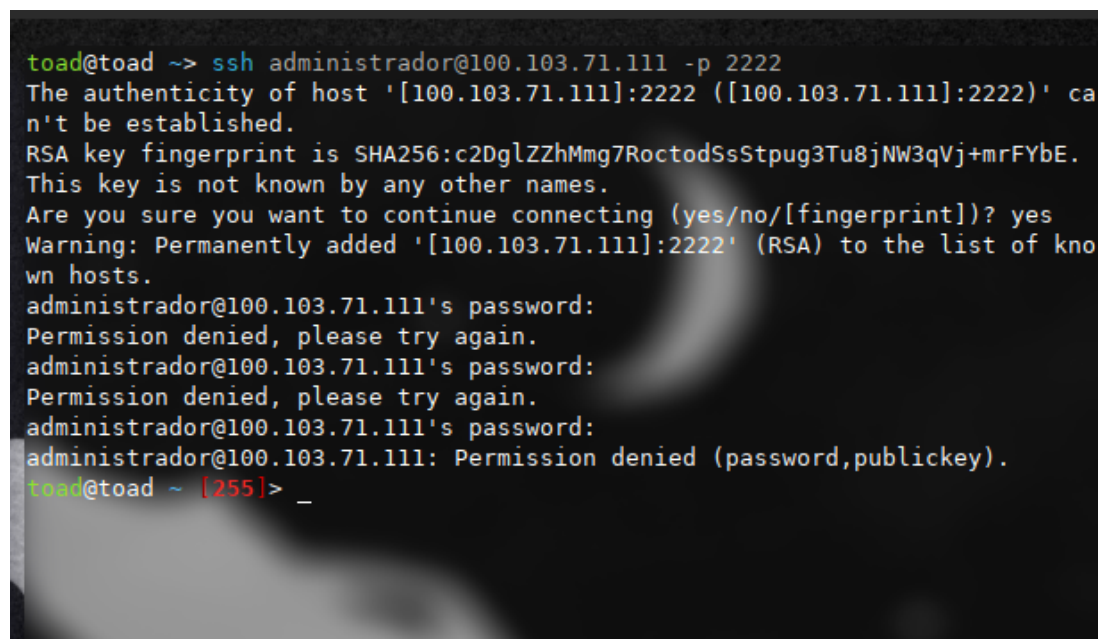
INFO | __main__:start:208 - ♦ SSH Honeypot iniciado en 0.0.0.0:2222 (Banner: SSH-2.0-OpenSSH_7.4)
```

Con el entorno virtual activado, se ejecuta `python3 ssh_trap.py`. La salida de logs muestra la secuencia de inicio:

- **HONEYPOT-LAB — SSH HONEYPOT v0.1 (CON BD)**
- Host: 0.0.0.0, Puerto: 2222, Banner: SSH-2.0-OpenSSH_7.4, Máx. intentos de autenticación: 3
- Conexión exitosa a la BD: localhost:5432/honeypot_db
- Tabla attacks inicializada correctamente
- **SSH Honeypot iniciado en 0.0.0.0:2222**

El honeypot queda en escucha en el puerto 2222 de todas las interfaces de red.

6.3 Intento de conexión SSH desde el atacante



```
toad@toad ~-> ssh administrador@100.103.71.111 -p 2222
The authenticity of host '[100.103.71.111]:2222 ([100.103.71.111]:2222)' ca
n't be established.
RSA key fingerprint is SHA256:c2DglZZhMmg7RoctodSsStpug3Tu8jNW3qVj+mrFYbE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[100.103.71.111]:2222' (RSA) to the list of kno
wn hosts.
administrador@100.103.71.111's password:
Permission denied, please try again.
administrador@100.103.71.111's password:
Permission denied, please try again.
administrador@100.103.71.111's password:
administrador@100.103.71.111: Permission denied (password,publickey).
toad@toad ~ [255]> _
```

Desde una máquina externa (toad@toad) se ejecuta ssh administrador@100.103.71.111 -p 2222. El cliente SSH muestra la huella RSA del servidor honeypot (SHA256:c2DglZZhMmg7RoctodSsStpug3Tu8jNW3qVj+mrFYbE) y pregunta si se desea continuar — se acepta. Se realizan **3 intentos de contraseña**, todos respondidos con Permission denied. Finalmente el cliente recibe Permission denied (password,publickey) y la conexión se cierra. El atacante no obtiene acceso al sistema real.

6.4 Captura y clasificación del ataque SSH

```
(honeypot-env) administrador@honeypot:~/honeypot-lab$ source ~/honeypot-env/bin/activate
(honeypot-env) administrador@honeypot:~/honeypot-lab$ python3 ssh_trap.py
INFO | __main__:main:295 - =====
INFO | __main__:main:296 - HONEYPOT-LAB - SSH HONEYPOT v0.1 (CON BD)
INFO | __main__:main:297 - =====
INFO | __main__:main:298 - Inicializando SSH Honeypot...
INFO | __main__:main:299 - Host: 0.0.0.0
INFO | __main__:main:300 - Puerto: 2222
INFO | __main__:main:301 - Banner: SSH-2.0-OpenSSH_7.4
INFO | __main__:main:302 - Max intentos auth: 3
INFO | __main__:main:303 - =====
INFO | __main__:main:306 -
♦ Inicializando base de datos...
INFO | database:connect:39 - ♦ Conexión a BD exitosa: localhost:5432/honeypot_db
INFO | database:init_database:408 - ♦ Tabla 'attacks' inicializada correctamente
INFO | logger:log_db_event:50 - Tabla inicializada | attacks
INFO | __main__:main:309 - ♦ Base de datos lista

INFO | __main__:start:208 - ♦ SSH Honeypot iniciado en 0.0.0.0:2222 (Banner: SSH-2.0-OpenSSH_7.4)
INFO | __main__:accept_connections:234 - ♦ Cliente #1 conectado: 100.111.145.79
INFO | logger:log_connection:46 - Conexión | IP: 100.111.145.79 | Servicio: SSH | Puerto: 2222
INFO | __main__:__init__:45 - ♦ Conexión SSH recibida desde 100.111.145.79:60888
WARNING | __main__:check_auth_password:56 - ♦ SSH LOGIN ATTEMPT | IP: 100.111.145.79 | Username: administrador | Pass
INFO | database:insert_attack:147 - ♦ Ataque insertado | ID: 1 | IP: 100.111.145.79 | Tipo: LOGIN_ATTEMPT
INFO | __main__:check_auth_password:90 - ♦ Ataque guardado en BD | ID: 1
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 100.111.145.79 | Servicio: SSH | Tipo: LOGIN_ATTEMPT
WARNING | __main__:check_auth_password:56 - ♦ SSH LOGIN ATTEMPT | IP: 100.111.145.79 | Username: administrador | Pass
INFO | database:insert_attack:147 - ♦ Ataque insertado | ID: 2 | IP: 100.111.145.79 | Tipo: BRUTE_FORCE
INFO | __main__:check_auth_password:90 - ♦ Ataque guardado en BD | ID: 2
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 100.111.145.79 | Servicio: SSH | Tipo: BRUTE_FORCE
WARNING | __main__:check_auth_password:56 - ♦ SSH LOGIN ATTEMPT | IP: 100.111.145.79 | Username: administrador | Pass
INFO | database:insert_attack:147 - ♦ Ataque insertado | ID: 3 | IP: 100.111.145.79 | Tipo: BRUTE_FORCE
INFO | __main__:check_auth_password:90 - ♦ Ataque guardado en BD | ID: 3
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 100.111.145.79 | Servicio: SSH | Tipo: BRUTE_FORCE
CRITICAL | __main__:check_auth_password:106 - ♦ BRUTE FORCE CRÍTICO DETECTADO | IP: 100.111.145.79 | Intentos: 3
```

En el terminal del honeypot se registra en tiempo real el ataque procedente de 100.111.145.79:

- **Cliente #1 conectado** — se recibe la conexión en el puerto 2222.
 - **ID 1** — primer intento: LOGIN_ATTEMPT, usuario administrador — insertado en BD.
 - **ID 2** — segundo intento: clasificado como BRUTE_FORCE — guardado en BD.
 - **ID 3** — tercer intento: BRUTE_FORCE — guardado en BD.
 - Nivel **CRITICAL**: BRUTE FORCE CRÍTICO DETECTADO | IP: 100.111.145.79 | Intentos: 3 — se eleva la alerta al alcanzar el umbral máximo de intentos.
-

6.5 Estadísticas de ataques SSH

```
=====
ESTADISTICAS
=====
INFO      | database:get_attack_stats:328 - 📊 Estadísticas obtenidas | Total: 3 ataques

Total ataques: 3

Por servicio:
- SSH: 3 ataques

Por tipo:
- BRUTE_FORCE: 2 ataques
- LOGIN_ATTEMPT: 1 ataques

=====
(honeypot-env) administrador@honeypot:~/honeypot-labs
```

El módulo de estadísticas muestra el resumen de los ataques registrados hasta ese momento en la base de datos:

- **Total ataques: 3**
 - Por servicio — SSH: 3 ataques
 - Por tipo — BRUTE_FORCE: 2 ataques, LOGIN_ATTEMPT: 1 ataque
-

7. HTTP Honeypot

7.1 Código del módulo http_trap.py

```
GNU nano 7.2 http_trap.py
#####
HTTP Honeypot - Captura ataques web
Detecta SQL injection, XSS, path traversal y otros ataques HTTP
Guarda todos los ataques en PostgreSQL
#####

import socket
import threading
import time
import os
import sys
from datetime import datetime
from urllib.parse import unquote

# Agregar parent directory al path
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from logger import logger, log_attack, log_connection, log_error
import config
import database

class HTTPHoneypot:
    """
    Servidor HTTP honeypot que captura ataques web
    """

    def __init__(self, host=config.HONEYPOT_HOST, port=config.HONEYPOT_HTTP_PORT):
        self.host = host
        self.port = port
        self.running = False
        self.server_socket = None
        self.request_count = 0

    def start(self):
        """Iniciar servidor HTTP"""
        try:
            self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

            self.server_socket.bind((self.host, self.port))
            self.server_socket.listen(100)
            self.running = True

            logger.info(f"HTTP Honeypot iniciado en {self.host}:{self.port}")
        except Exception as e:
            logger.error(f"Error al iniciar el servidor HTTP: {e}")
```

Se muestra el fichero **http_trap.py** en el editor nano. El módulo implementa la clase **HTTPHoneypot**, que levanta un servidor HTTP usando sockets de bajo nivel (`socket.AF_INET`, `socket.SOCK_STREAM`). El servidor escucha en el host y puerto configurados (por defecto el `HONEYPOT_HTTP_PORT` del módulo `config`), admite hasta 100 conexiones en cola y registra cada petición entrante. Su descripción indica que detecta **SQL injection**, **XSS**, **path traversal** y otros ataques HTTP, guardándolos en PostgreSQL.

7.2 Arranque del HTTP Honeypot

```
(honeypot-env) administrador@honeypot:~/honeypot-lab$ source ~/honeypot-env/bin/activate
(honeypot-env) administrador@honeypot:~/honeypot-lab$ python3 http_trap.py
INFO | __main__:main:219 - =====
INFO | __main__:main:220 - HONEYPOT-LAB - HTTP HONEYPOT v0.1
INFO | __main__:main:221 - =====
INFO | __main__:main:222 - Inicializando HTTP Honeypot...
INFO | __main__:main:223 - Host: 0.0.0.0
INFO | __main__:main:224 - Puerto: 8080
INFO | __main__:main:225 - =====
INFO | __main__:main:228 -
Verificando base de datos...
INFO | database:connect:39 - ♦ Conexión a BD exitosa: localhost:5432/honeypot_db
INFO | database:init_database:408 - ♦ Tabla 'attacks' inicializada correctamente
INFO | logger:log_db_event:50 - Tabla inicializada | attacks
INFO | __main__:main:231 - Base de datos lista

INFO | __main__:start:45 - HTTP Honeypot iniciado en 0.0.0.0:8080
```

Con el entorno virtual activado se ejecuta `python3 http_trap.py`. La secuencia de inicio es:

- **HONEYPOT-LAB — HTTP HONEYPOT v0.1**
- Host: 0.0.0.0, Puerto: 8080
- Conexión exitosa a la BD: localhost:5432/honeypot_db
- Tabla attacks inicializada correctamente
- **HTTP Honeypot iniciado en 0.0.0.0:8080**

El servidor queda a la escucha en el puerto 8080.

7.3 Ataque XSS manual con curl

```
administrador@honeypot:~/honeypot-lab$ curl "http://localhost:8080/search?q=<script>alert(1)</script>"
<html><body><h1>Welcome</h1></body></html>administrador@honeypot:~/honeypot-lab$ _
```

Desde la misma máquina se lanza manualmente un ataque XSS usando curl:

```
curl "http://localhost:8080/search?q=<script>alert(1)</script>"
```

El honeypot responde con `<html><body><h1>Welcome</h1></body></html>`, simulando ser un servidor web legítimo para no delatar que es una trampa. El atacante recibe una respuesta HTTP 200 aparentemente normal.

7.4 Detección y registro del ataque XSS

```
(honeypot-env) administrador@honeypot:~/honeypot-lab$ source ~/honeypot-env/bin/activate
(honeypot-env) administrador@honeypot:~/honeypot-lab$ python3 http_trap.py
INFO      | __main__:main:219 - =====
INFO      | __main__:main:220 - HONEYPOT-LAB - HTTP HONEYPOT v0.1
INFO      | __main__:main:221 - =====
INFO      | __main__:main:222 - Inicializando HTTP Honeypot...
INFO      | __main__:main:223 - Host: 0.0.0.0
INFO      | __main__:main:224 - Puerto: 8080
INFO      | __main__:main:225 - =====
INFO      | __main__:main:226 - =====
INFO      | __main__:main:227 - =====
INFO      | __main__:main:228 - =====
Verificando base de datos...
INFO      | database:connect:39 - ♦ Conexión a BD exitosa: localhost:5432/honeypot_db
INFO      | database:init_database:408 - ♦ Tabla 'attacks' inicializada correctamente
INFO      | logger:log_db_event:50 - Tabla inicializada | attacks
INFO      | __main__:main:231 - Base de datos lista

INFO      | __main__:start:45 - HTTP Honeypot iniciado en 0.0.0.0:8080
INFO      | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /search?q=<script>alert(1)</script>
INFO      | database:insert_attack:147 - ♦ Ataque insertado | ID: 4 | IP: 127.0.0.1 | Tipo: SQL_INJECTION
INFO      | __main__:handle_request:127 - Ataque HTTP guardado | ID: 4 | Tipo: SQL_INJECTION
WARNING   | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: SQL_INJECTION
```

El honeypot registra internamente la petición:

- **HTTP Request** — IP: 127.0.0.1 | GET /search?q=<script>alert(1)</script>
- Ataque insertado en BD con **ID: 4**, tipo clasificado como **SQL_INJECTION**
- **WARNING: ATAQUE DETECTADO** | IP: 127.0.0.1 | Servicio: HTTP | Tipo: SQL_INJECTION

El payload XSS es detectado aunque el motor de clasificación lo etiqueta como SQL_INJECTION, lo que indica que el patrón de detección agrupa ambos tipos bajo la misma categoría en esta versión.

8. Simulador de ataques

8.1 Código del simulador — simulate_atacks.py

```
GNU nano 7.2 simulate_atacks.py
#####
Simulador de ataques para honeypot-lab
Genera ataques SSH y HTTP para testing
#####

import socket
import time
import random
import requests
from datetime import datetime

# Configuración
SSH_HOST = "localhost"
SSH_PORT = 2222
HTTP_HOST = "localhost"
HTTP_PORT = 8080

# Datos de prueba
SSH_USERS = [
    "admin", "root", "test", "user", "administrator",
    "postgres", "mysql", "oracle", "ubuntu", "pi"
]

SSH_PASSWORDS = [
    "admin", "password", "123456", "password123", "admin123",
    "root", "12345678", "qwerty", "test", "letmein"
]

HTTP_PAYLOADS = {
    "SQL_INJECTION": [
        "/index.php?id=1' OR '1'='1",
        "/login.php?user=admin'--",
        "/search?q='; DROP TABLE users--",
        "/api/users?id=1 UNION SELECT * FROM passwords",
        "/product.php?id=1' AND 1=1--"
    ],
    "XSS": [
        "/search?q=<script>alert(1)</script>",
        "/comment?text=<img src=x onerror=alert(1)>",
        "/page?name=<script>document.cookie</script>",
        "/input?data=<iframe src=evil.com>",
        "/search?q=<svg onload=alert(1)>"
    ],
    "PATH_TRAVERSAL": [
        "/download?file=../../../../etc/passwd",
        "/read?path=../../../../etc/shadow"
    ]
}

[ Read 191 lines ]
Help Write Out Where Is Cut Execute Location M-U Undo M-A Se
Exit Read File Replace Paste Justify Go To Line M-E Redo M-B Co
```

Se muestra el fichero **simulate_atacks.py** en nano. El simulador apunta a localhost en los puertos 2222 (SSH) y 8080 (HTTP). Define diccionarios de prueba para SSH: lista de usuarios comunes (admin, root, test, administrator, postgres, ubuntu...) y contraseñas débiles (admin, password, 123456, qwerty...). Para HTTP define payloads organizados por categoría: **SQL_INJECTION** (inyecciones clásicas con OR '1'='1', UNION SELECT, DROP TABLE), **XSS** (payloads <script>, , <svg onload>), **PATH_TRAVERSAL** (../../../../etc/passwd, ../../etc/shadow) y peticiones a rutas sensibles de **SCANNER**.

8.2 Menú del simulador de ataques

```
administrador@honeypot:~/honeypot-lab/tests$ cd
administrador@honeypot:~$ cd honeypot-lab/
administrador@honeypot:~/honeypot-lab$ source ~/honeypot-env/bin/activate
(honeypot-env) administrador@honeypot:~/honeypot-lab$ python3 tests/simulate_atacks.py
=====
HONEYPOT-LAB - SIMULADOR DE ATAQUES
=====

Opciones:
1. Simular ataque SSH (fuerza bruta)
2. Simular ataques HTTP (SQL, XSS, etc)
3. Simular ataques mixtos (SSH + HTTP)
4. Ataque rapido (5 SSH + 10 HTTP)
5. Estres test (100 requests)
0. Salir

Selecciona opcion: _
```

Se ejecuta `python3 tests/simulate_atacks.py` desde el directorio `~/honeypot-lab`. El simulador presenta un menú interactivo con las siguientes opciones:

1. Simular ataque SSH (fuerza bruta)
 2. Simular ataques HTTP (SQL, XSS, etc.)
 3. Simular ataques mixtos (SSH + HTTP)
 4. Ataque rápido (5 SSH + 10 HTTP)
 5. Estrés test (100 requests)
 6. Salir
-

8.3 Ejecución del ataque rápido (opción 4)

```
Opciones:
1. Simular ataque SSH (fuerza bruta)
2. Simular ataques HTTP (SQL, XSS, etc)
3. Simular ataques mixtos (SSH + HTTP)
4. Ataque rapido (5 SSH + 10 HTTP)
5. Estres test (100 requests)
6. Salir

Selecciona opción: 4

[QUICK] Ejecutando ataque rapido...

[SSH] Iniciando simulacion de 5 intentos de login...
Error en intento 1: [Errno 111] Connection refused
Error en intento 2: [Errno 111] Connection refused
Error en intento 3: [Errno 111] Connection refused
Error en intento 4: [Errno 111] Connection refused
Error en intento 5: [Errno 111] Connection refused
[SSH] Simulacion completada: 5 intentos

[HTTP] Iniciando simulacion de 10 requests...
Error en request 1: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /search?q=%3Csvg%3E
ConnectionError('HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused')
Error en request 2: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /api/users?id=15
ConnectionError('HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused')
Error en request 3: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /view?page=...
ConnectionError('HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused')
Error en request 4: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /read?path=...
ConnectionError('HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused')
Error en request 5: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /get?file=.%2F.
ConnectionError('HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused')
Error en request 6: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /input?data=%3C%3E
ConnectionError('HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused')
Error en request 7: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /get?file=.%2F.
ConnectionError('HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused')
Error en request 8: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /comment?text=%
ConnectionError('HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused')
Error en request 9: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /wp-admin (Cause
ConnectionError('HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused')
Error en request 10: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /phpmyadmin (Ca
ConnectionError('HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused')

[HTTP] Simulacion completada:
- SQL_INJECTION: 0 requests
- XSS: 0 requests
- PATH_TRAVERSAL: 0 requests
- SCANNER: 0 requests

[QUICK] Ataque rapido completado
(honey-pot-env) administrador@honeypot:~/honeypot-lab$ _
```

Se selecciona la opción 4 — **Ataque rápido**. El simulador lanza 5 intentos SSH y 10 peticiones HTTP. En esta ejecución ambos servicios responden con **[Errno 111] Connection refused** porque los honeypots no estaban en ejecución en ese momento. El simulador registra la simulación como completada mostrando: SQL_INJECTION: 0 requests, XSS: 0 requests, PATH_TRAVERSAL: 0 requests, SCANNER: 0 requests.

8.4 HTTP Honeypot procesando múltiples ataques simulados

```
INFO | __main__:main:225 - =====
INFO | __main__:main:228 -
Verificando base de datos...
INFO | database:connect:39 - * Conexión a BD exitosa: localhost:5432/honeypot_db
INFO | database:init_database:408 - * Tabla 'attacks' inicializada correctamente
INFO | logger:log_db_event:50 - Tabla inicializada | attacks
INFO | __main__:main:231 - Base de datos lista

INFO | __main__:start:45 - HTTP Honeypot iniciado en 0.0.0.0:8080
INFO | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /read?path=../../etc/shadow
INFO | database:insert_attack:147 - * Ataque insertado | ID: 6 | IP: 127.0.0.1 | Tipo: PATH_TRAVERSAL
INFO | __main__:handle_request:127 - Ataque HTTP guardado | ID: 6 | Tipo: PATH_TRAVERSAL
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: PATH_TRAVERSAL
INFO | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /page?name=<script>document.cookie</scri
INFO | database:insert_attack:147 - * Ataque insertado | ID: 7 | IP: 127.0.0.1 | Tipo: SQL_INJECTION
INFO | __main__:handle_request:127 - Ataque HTTP guardado | ID: 7 | Tipo: SQL_INJECTION
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: SQL_INJECTION
INFO | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /search?q=<svg onload=alert(1)>
INFO | database:insert_attack:147 - * Ataque insertado | ID: 8 | IP: 127.0.0.1 | Tipo: SQL_INJECTION
INFO | __main__:handle_request:127 - Ataque HTTP guardado | ID: 8 | Tipo: SQL_INJECTION
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: SQL_INJECTION
INFO | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /read?path=../../etc/shadow
INFO | database:insert_attack:147 - * Ataque insertado | ID: 9 | IP: 127.0.0.1 | Tipo: PATH_TRAVERSAL
INFO | __main__:handle_request:127 - Ataque HTTP guardado | ID: 9 | Tipo: PATH_TRAVERSAL
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: PATH_TRAVERSAL
INFO | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /search?q=<script>alert(1)</script>
INFO | database:insert_attack:147 - * Ataque insertado | ID: 10 | IP: 127.0.0.1 | Tipo: SQL_INJECTION
INFO | __main__:handle_request:127 - Ataque HTTP guardado | ID: 10 | Tipo: SQL_INJECTION
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: SQL_INJECTION
INFO | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /admin
INFO | database:insert_attack:147 - * Ataque insertado | ID: 11 | IP: 127.0.0.1 | Tipo: SCANNER
INFO | __main__:handle_request:127 - Ataque HTTP guardado | ID: 11 | Tipo: SCANNER
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: SCANNER
INFO | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /.env
INFO | database:insert_attack:147 - * Ataque insertado | ID: 12 | IP: 127.0.0.1 | Tipo: SCANNER
INFO | __main__:handle_request:127 - Ataque HTTP guardado | ID: 12 | Tipo: SCANNER
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: SCANNER
INFO | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /view?page=../../../config.php
INFO | database:insert_attack:147 - * Ataque insertado | ID: 13 | IP: 127.0.0.1 | Tipo: PATH_TRAVERSAL
INFO | __main__:handle_request:127 - Ataque HTTP guardado | ID: 13 | Tipo: PATH_TRAVERSAL
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: PATH_TRAVERSAL
INFO | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /product.php?id=1' AND 1=1--
INFO | database:insert_attack:147 - * Ataque insertado | ID: 14 | IP: 127.0.0.1 | Tipo: SUSPICIOUS_REQUEST
INFO | __main__:handle_request:127 - Ataque HTTP guardado | ID: 14 | Tipo: SUSPICIOUS_REQUEST
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: SUSPICIOUS_REQUEST
INFO | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /search?q=<svg onload=alert(1)>
INFO | database:insert_attack:147 - * Ataque insertado | ID: 15 | IP: 127.0.0.1 | Tipo: SQL_INJECTION
INFO | __main__:handle_request:127 - Ataque HTTP guardado | ID: 15 | Tipo: SQL_INJECTION
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: SQL_INJECTION
```

Con el HTTP Honeypot activo, el simulador genera una ráfaga de peticiones maliciosas desde 127.0.0.1. El honeypot detecta y registra en BD los siguientes ataques en secuencia:

ID	Tipo	Ruta
6	PATH_TRAVERSAL	/read?path=../../etc/shadow
7	SQL_INJECTION	/page?name=<script>document.cookie</script>
8	SQL_INJECTION	/search?q=<svg onload=alert(1)>
9	PATH_TRAVERSAL	/read?path=../../etc/shadow
10	SQL_INJECTION	/search?q=<script>alert(1)</script>
11	SCANNER	/admin
12	SCANNER	/.env
13	PATH_TRAVERSAL	/view?page=../../../config.php
14	SUSPICIOUS_REQUEST	/product.php?id=1' AND 1=1--

ID	Tipo	Ruta
15	SQL_INJECTION	/search?q=<svg onload=alert(1)>

8.5 SSH Honeypot recibiendo conexiones del simulador

El SSH Honeypot recibe conexiones desde 127.0.0.1 generadas por el simulador. Los clientes #4 y #5 se conectan al puerto 2222 pero no completan el handshake SSH correctamente, produciendo la excepción `paramiko.ssh_exception.SSHException: Error reading SSH protocol banner`. Esta excepción indica que la conexión se establece a nivel TCP pero el cliente (el simulador) no envía el banner SSH estándar, lo que interrumpe el intercambio del protocolo. El honeypot registra cada conexión entrante antes de producirse el error.